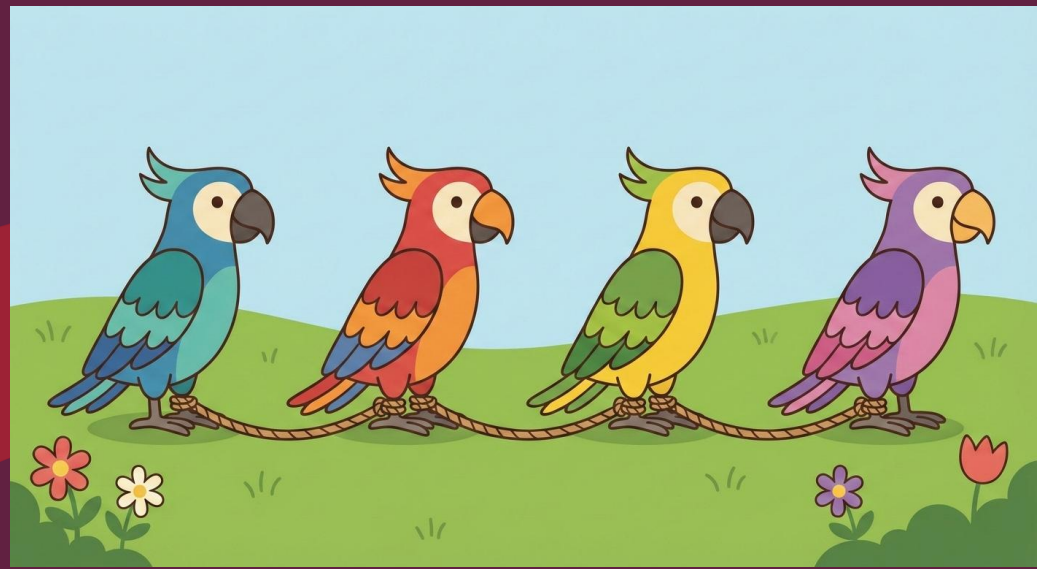
Agentic AI with LangChain and LangGraph



Presentation 6: Intro to LangGraph
Primary Instructor: Michael Z
RA: YH
Date: 2026.Jan-Feb

Sobey School of Business
Saint Mary's University
MBAN

Saint Mary's University

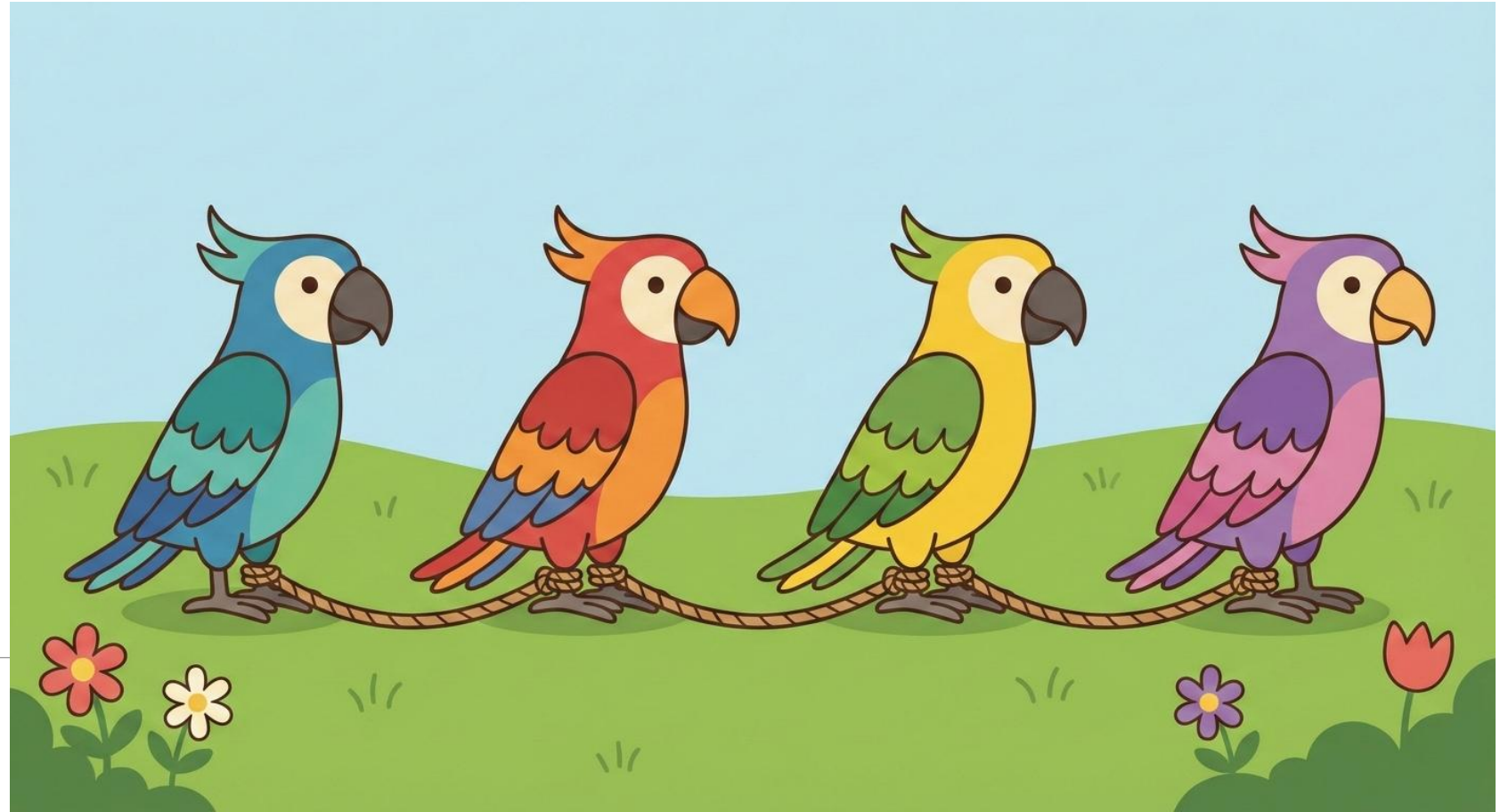
0. I wish you did the QuickStart of LangGraph (Graph API only). You can do it now if you didn’t

1. Think in Graph

1. Think in Graph: Let’s chain these parrots together

Still remember the interesting explanation of LangChain logo? Unfortunately, LangChain, the Python package, alone, can’t achieve what LangChain logo means to do. LangGraph is the one you would use to chain these parrots (LLMs) together.

What are the differences between LangChain and LangGraph? Why LangGraph is a separate package, rather than a part of LangChain?



1. Think in Graph: Let’s think of problem step by step

When we face a problem, what’s the commonly-used pattern of thinking of solution?

For example, how to make a bowl of salad?

First, you might buy the vegetables and eggs and sauces. Second, you boil the eggs. Third, you would cut the vegetables…

You solve the problem **STEP BY STEP**.
In LangGraph, the step we talked about here is a node. An edge would decide what’s the next node to call.
The state = data structure (schema) + reducer (might not talk about it in this lecture), simply.

Designing multi-agent is nothing but designing a Python script to solve a problem step by step, with some logics and engineering details added, using LangGraph.

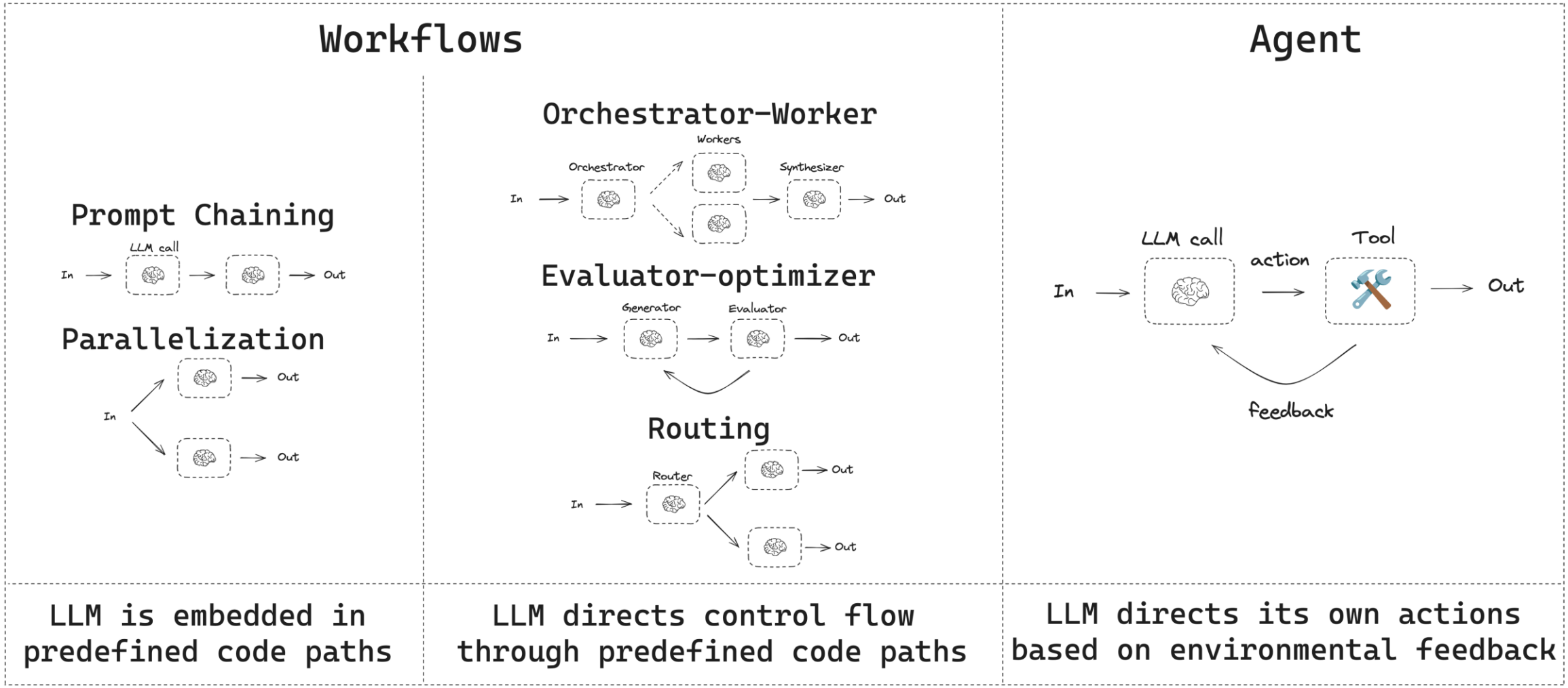
A node is basically a function and an edge is a bunch of if else bundled to decide what’s next.

If something could be done by plain LLM or agent in this node, you can simply call it multi-agent.

Actually, you could put whatever you like in a node.

1. Think in Graph: Workflow, Agentic Workflow, Agent

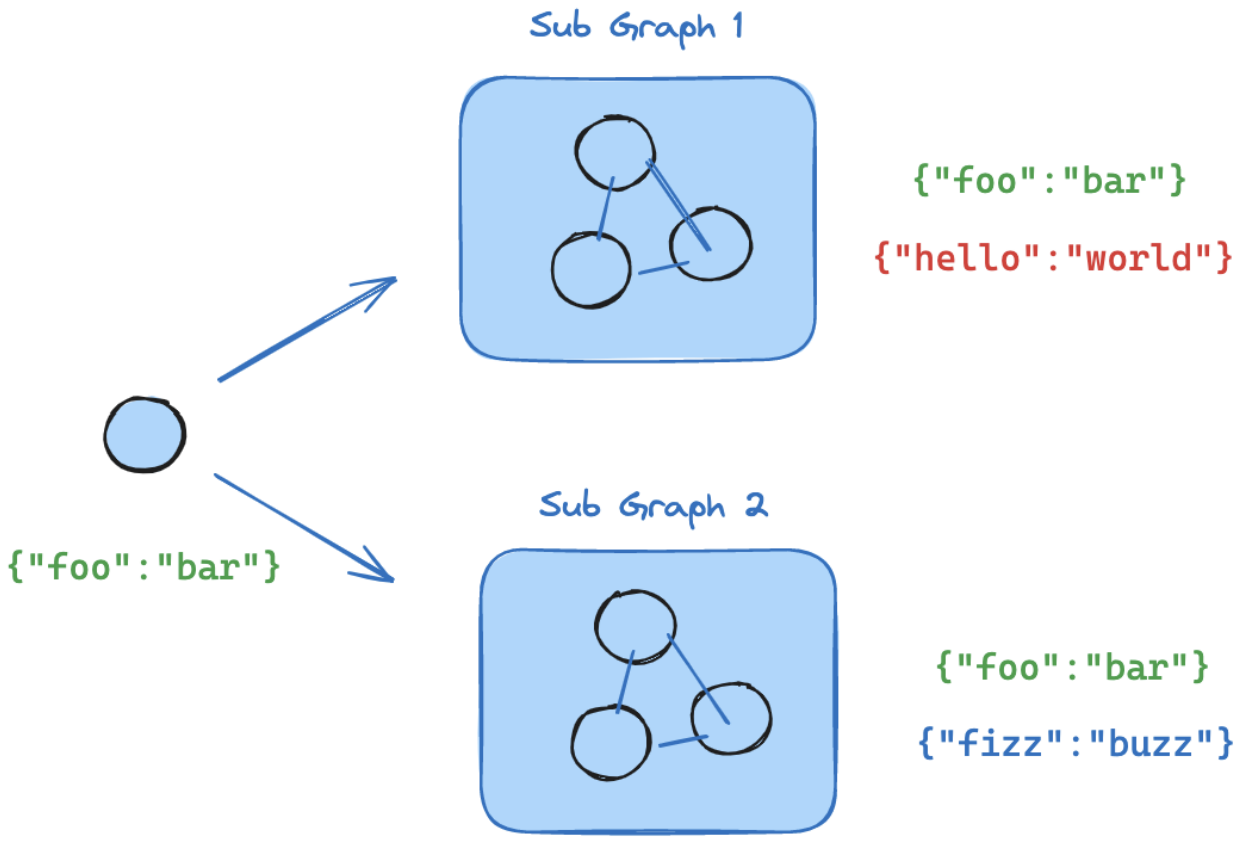
I love this image cause it basically tells everything we need to know.



Source: https://docs.langchain.com/oss/python/langgraph/workflows-agents

1. Think in Graph: SubGraph as a Node

Just simply invoke a pre-compiled graph in a node…….




```python
class SubgraphState(TypedDict):
    bar: str

# Subgraph

def subgraph_node_1(state: SubgraphState):
    return {"bar": "hi! " + state["bar"]}

subgraph_builder = StateGraph(SubgraphState)
subgraph_builder.add_node(subgraph_node_1)
subgraph_builder.add_edge(START, "subgraph_node_1")
subgraph = subgraph_builder.compile()

# Parent graph

class State(TypedDict):
    foo: str

def call_subgraph(state: State):
    # Transform the state to the subgraph state
    subgraph_output = subgraph.invoke({"bar": state["foo"]})
    # Transform response back to the parent state
    return {"foo": subgraph_output["bar"]}

builder = StateGraph(State)
builder.add_node("node_1", call_subgraph)
builder.add_edge(START, "node_1")
graph = builder.compile()
```

2. Graph API

2. Graph API: Nodes, Edges, Compilation

Go over the docs with you very quick.

Graph API basically consist of three components: Adding nodes (optionally with new name), Adding (conditional or non-conditional) edges (START & END are strings so no worries) and Compilation.

I would not discuss reducer in detail since how to update state could be done within node.

2. Graph API: Let’s go over a comprehensive and practical example

https://docs.langchain.com/oss/python/langchain/multi-agent/router-knowledge-base

3. Super Step

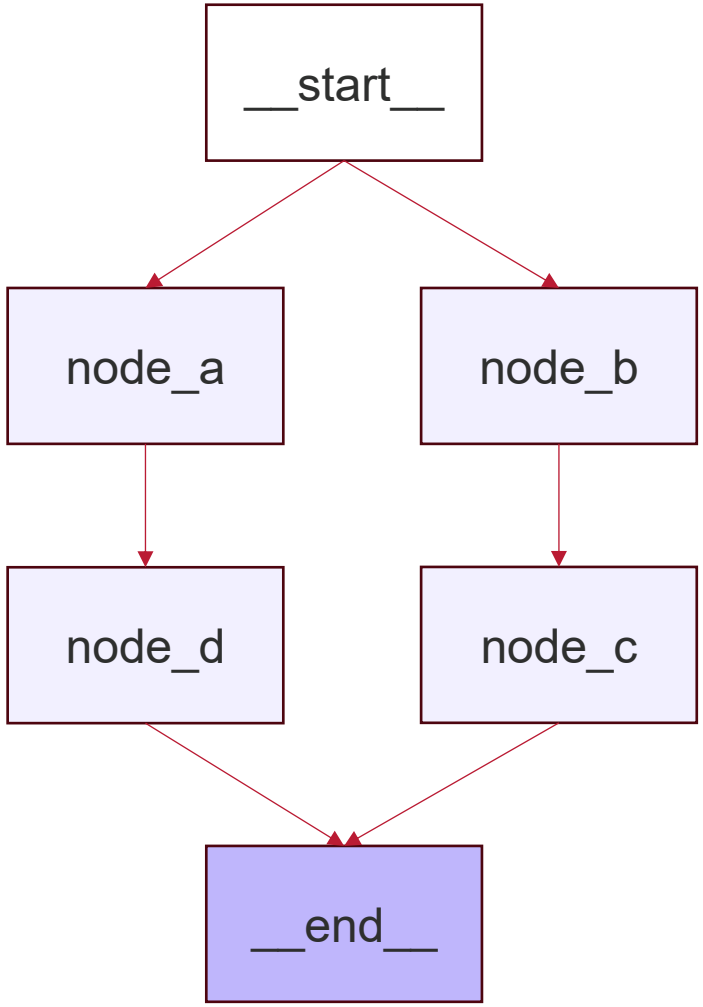
3. Super Step

This workflow works in parallel after START node receives input.

For example, if the custom state has only one field “user_input” and the input is {“user_input”: “start: ”}
Then, what node_a does is to append “a” to the “user_input” field of state. Similarly, node_b would update state by appending “b” to “user_input” of state.

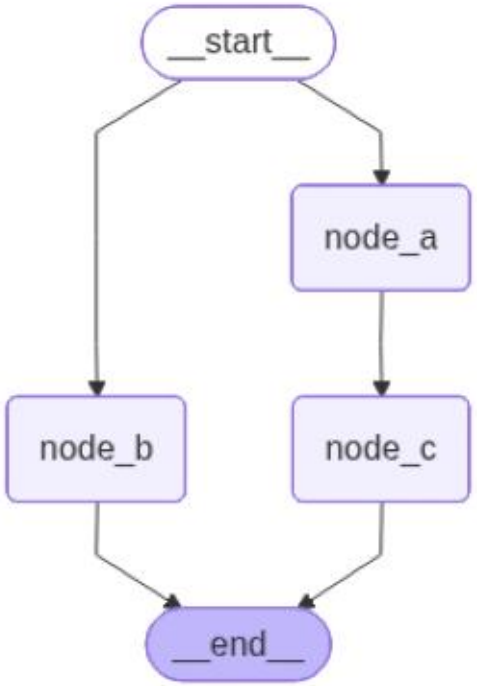
What node_c would see after it receives state from node_b?

A. {“user_input”: “start: a”}
B. {“user_input”: “start: b”}
C. {“user_input”: “start: ab”} or {“user_input”: “start: ba”}



3. Super Step: Trickier one

What about this one?



What c could see after it receives state from node_a?

```
what a can see is start:
what b can see is start:
what c can see is start: ab
----------
start: abc

Process finished with exit code 0
```

```python
import io
from operator import add
from typing import Annotated
from PIL import Image
from langgraph.constants import START, END
from langgraph.graph import StateGraph
from matplotlib import pyplot as plt
from typing_extensions import TypedDict
class CustomState(TypedDict):  4 usages
    user_input:Annotated[str, add]
def node_a(state: CustomState):  1 usage
    print(f"what a can see is {state['user_input']}")
    return {"user_input": "a"}
def node_b(state: CustomState):  1 usage
    print(f"what b can see is {state['user_input']}")
    return {"user_input": "b"}
def node_c(state: CustomState):  1 usage
    print(f"what c can see is {state["user_input"]}")
    return {"user_input": "c"}
builder = StateGraph(CustomState)
builder.add_node(node_a)
builder.add_node(node_b)
builder.add_node(node_c)
builder.add_edge(START,  end_key: "node_a")
builder.add_edge(START,  end_key: "node_b")
builder.add_edge( start_key: "node_a",  end_key: "node_c")
builder.add_edge( start_key: "node_c", END)
builder.add_edge( start_key: "node_b", END)
w = builder.compile()
result = w.invoke({"user_input": "start: "})
...
print("-"*10)
print(result["user_input"])
```

4. Hands-On Project Again

4. Hands-On Project: I want to see a lot of different designs

I know for a given task, there would be one or several designs of agentic workflow or workflow or multi-agent to accomplish. You are expected to create your own one and unleash your creativity.

Project Description: Make an agentic workflow or multi-agent application that emails Michael to express your interests of doing PhD with him.

At least, you need to create node/agent to search Michael's academic information and research interests and then create nodes/agent to write emails, include HITL to request your explicit approval before sending emails. Bs4 could help again.

Since there is a lot of ways to do it, you have enough flexibility and freedom.

Use gmail to send email in Python because it's much easier.

Don't actually send the emails to Michael. Send the emails to me for testing (Yilin.Huang@smu.ca).

Michael Zhang

FOLLOW

Saint Mary's University
Verified email at smu.ca - Homepage
AI for Business and Health...

TITLE	CITED BY	YEAR
Game theoretical perspectives on dual-channel supply chain competition with price discounts and pricing schemes GG Cai, ZG Zhang, M Zhang International Journal of Production Economics 117 (1), 80-96	579	2009
Price fencing in the practice of revenue management: An overview and taxonomy M Zhang, P Bell Journal of Revenue and Pricing Management 11 (2), 146-159	90	2012
Optimal fences and joint price and inventory decisions in distinct markets with demand leakage M Zhang, PC Bell, GG Cai, X Chen European Journal of Operational Research 204 (3), 589-596	79	2010
Optimal decisions in a retailer Stackelberg supply chain X Chen, H Zhang, M Zhang, J Chen International Journal of Production Economics 187, 260-270	76	2017
The effect of market segmentation with demand leakage between market segments on a firm's price and inventory decisions M Zhang, PC Bell European Journal of Operational Research 182 (2), 738-754	74	2007
Optimal pricing and production strategies for new and remanufactured products under a non-renewing free replacement warranty Z Liu, C Diallo, J Chen, M Zhang International Journal of Production Economics 226, 107602	71	2020
The impact of transitions, a mental health literacy intervention with embedded life skills for postsecondary students: preliminary findings from a naturalistic cohort study Y Wei, S Kutcher, E Austen, A Comfort, C Gilham, C MacDougall, ... The Canadian Journal of Psychiatry 67 (6), 452-461	32	2022
Fencing in the context of revenue management M Zhang, PC Bell	32	2010

Sobey School of Business
Saint Mary's University
MBAN

*Quick Notes

```python
import os
import smtplib
import ssl
from email.message import EmailMessage
sender_email = "***********"
receiver_email = "********"
subject = "An Email from Python"
body = ""
# Create the email message object
msg = EmailMessage()
msg.set_content(body)
msg['Subject'] = subject
msg['From'] = sender_email
msg['To'] = receiver_email
smtp_server = "smtp.gmail.com"
port = 465 # For SSL connection
app_password = os.getenv("APP_PASSWORD")
context = ssl.create_default_context()
try:
    with smtplib.SMTP_SSL(smtp_server, port, context=context) as server:
        server.login(sender_email, app_password)
        server.send_message(msg)
        print("Email sent successfully!")
except smtplib.SMTPException as e:
    print(f"Error: {e}")
```

Regarding the email sending, when I was troubleshooting for someone using MacBook on Thursday, I noticed that “ssl.create_default_context” should be replaced by “ssl._create_unverified_context”. Thanks to MR. A possibly helpful code snippet is attached on the LHS.

I wasn’t aware of Google Scholar’s blockage to web retrieval using bs4 and requests until Sunday, thanks to Dale. It’s suggested to use selenium to retrieve information about Dr. Michael from Google scholar.

A possibly helpful code snippet of selenium is uploaded in Brightspace. (You need to install selenium and webdriver-manager using uv)

To use gmail in Python, please create an app password of your Google account. Please be advised that 2-step verification needs to be turned on before creating any app passwords.

Clarifications: Screenshot of Dr. Michael’s Google scholar profile indicates the fact that Dr. Michael is a scholar. Dr. Michael’s academic information is also available in SMU website(s).

Thank you for your paged attention!

arXiv > cs > arXiv:2309.06180

Computer Science > Machine Learning

[Submitted on 12 Sep 2023]

Efficient Memory Management for Large Language Model Serving with PagedAttention

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, Ion Stoica